

# OneLP: AI Document-Intelligence and Portfolio-Forecasting for Limited Partners

## Capstone Written Report — Proof-of-Concept Evaluation

**Authors:** Kishan Dhulashia · Max Vanderlinden · Sofia Mollon · Agustin Gentil

**Programme:** MSc in Business Analytics & Big Data — IE School of Science & Technology (Venture Capstone)

**Date:** June 2026

**Code & notebooks:** <https://github.com/agentilt/onelp-capstone>

---

## Ethics Statement

This capstone was conducted in accordance with responsible data-science and AI practices. **No real pilot-client data or personally identifiable information was used in any evaluation.** The document-extraction test set is a synthetic, labelled corpus modelled on the project's own golden fixtures and schemas; the forecasting study uses a **calibrated reference portfolio** anchored on seeded demo accounts rather than confidential client holdings. Results derived from simulated or calibrated processes are explicitly labelled as such throughout, are distinguished from production behaviour, and are accompanied by a transparent account of limitations (calibration, cross-asset correlation fallback, cold-start, and generalisation across document formats). The underlying system retains a **human-in-the-loop review queue** for low-confidence extractions, so automated outputs never drive financial decisions without verification, and every extracted field keeps source-linked evidence for auditability. All metrics are computed from first principles with **deterministic, openly available code** to ensure reproducibility, and no claim is made beyond what the evaluation measures. The authors affirm that this work is their own and that all external sources and tools are duly acknowledged.

## OneLP Capstone — Proof-of-Concept Deliverables

### Executive summary

The OneLP capstone builds a production-grade **document-intelligence pipeline** that turns unstructured private-market documents into structured, validated, portfolio-ready data, and an **analytics layer** that forecasts portfolio cash flow with calibrated uncertainty. This document synthesizes the three Proof-of-Concept deliverables committed to in §7 of the proposal, each evaluated against the quantitative target it was set.

| Deliverable                    | Proposal target                                       | Result                                | Verdict |
|--------------------------------|-------------------------------------------------------|---------------------------------------|---------|
| <b>1 — Extraction accuracy</b> | Macro F1 $\geq 0.90$ ;<br>critical fields $\geq 0.95$ | Hybrid <b>0.968</b> /<br><b>0.968</b> | ✅ met   |
| <b>2 — Forecast</b>            | MAE $< 15\%$ of                                       | <b>5.0% MAE</b> (20-                  | ✅ met   |

| Deliverable                       | Proposal target      | Result               | Verdict                                                                                 |
|-----------------------------------|----------------------|----------------------|-----------------------------------------------------------------------------------------|
| <b>backtest</b>                   | quarterly cash flow  | point rolling)       |                                                                                         |
| <b>3 — Latency<br/>(Metric 3)</b> | P50 < 30s; P95 < 90s | <b>23.9s / 29.4s</b> |  met |

All three quantitative targets are met. Beyond the headline numbers, the deliverables surface three findings that matter for production: the **hybrid extraction architecture is empirically justified** (not just intuitive), the **forecast intervals are slightly too tight**, and the **cross-asset correlation model silently no-ops on realistic multi-fund books** — a concrete, fixable engineering gap.

## Deliverable 1 — Document Extraction Accuracy

A 54-document labeled test set across 8 GP document types is scored with the repository's own field-matching logic (extended to be numeric-aware for money/dates). The **hybrid** engine (Document AI + LLM, merged with precedence) is the **only** configuration meeting both accuracy bars:

- Macro F1 **0.968** ( $\geq 0.90$  target) · critical-field F1 **0.968** ( $\geq 0.95$  target)
- Document AI alone: strong on financial fields (0.963) but weak on narrative → 0.889 macro
- LLM alone: strong overall (0.892) but weakest on the critical financial fields (0.887)
- Document-type classification: **88.9% accuracy**, confusions only between adjacent periodic-report types

**Takeaway:** the two single engines fail in opposite directions; merging them recovers the best of both. The architecture bet pays off in the data.

## Deliverable 2 — Cash-Flow Forecasting with Backtesting

The production Monte Carlo engine (a faithful Python port of monte-carlo.ts) is run over a 16-fund / \$143M calibrated reference portfolio:

- **Backtest MAE 5.0%** of gross quarterly flow on a 20-point rolling-origin test — well inside the < 15% target. (80% predictive-interval coverage is 65% → bands are slightly too narrow.)
- **Stress testing** across 7 scenarios: cumulative net cash flow ranges from −\$6.8M (Liquidity Crisis) to +\$37.3M (Rapid Recovery); the implied 95% reserve scales with severity.
- **Liquidity reserve:** ~\$8M (95% confidence) needed to survive a Liquidity Crisis; the mature book self-funds in the base case.
- **Sensitivity:** NAV level ( $\pm \$16.8\text{M}$ ) and distribution pacing ( $\pm \$13.9\text{M}$ ) dominate; call timing matters  $\sim 3\times$  less.
- **Cross-asset correlation** widens the aggregate outcome distribution by +39% ( $\sigma$ ) on a one-per-class demo — but **does not engage on the full 16-fund book** because the per-

fund correlation matrix is singular and the engine falls back to independent simulation (a documented limitation, monte-carlo.ts:336-341).

**Takeaway:** the engine forecasts within target; the reserve sizing is the product's real value; two calibration/engineering fixes are identified (widen bands, regularize the correlation matrix).

### Deliverable 3 — Pipeline & Model Analysis

- **Confidence calibration:** ECE 0.056. The pipeline is **conservatively calibrated** — mean confidence 0.920 vs accuracy 0.965 (−4.5 pts) — it under-states its reliability, most in the structured categories (dates/tables/money) it handles near-perfectly. A small “confidently-wrong” tail is what the review queue must catch.
- **Threshold operating curve:** the per-type auto-apply thresholds (0.70–0.95) trade auto-apply rate against error leakage; at the 0.85 capital-call threshold, 90% of fields auto-apply at 1.9% leakage. Conservative calibration means thresholds could be relaxed after a calibration pass.
- **End-to-end latency (Metric 3):** P50 23.9s / P95 29.4s — both targets met; extraction dominates and is parallelized (cost = slower engine, not the sum).
- **Limitations:** data sparsity & cold-start (profile fallback until 8+ quarters of history), GP-format generalization (long tail of bespoke layouts), correlation-model fragility, and conservative calibration.

**Takeaway:** the pipeline's confidence is trustworthy and errs safe; the main opportunities are a calibration map (to reclaim review capacity) and a correlation-matrix fix.

---

### Cross-cutting conclusions

1. **The architecture choices are evidence-backed.** Hybrid extraction (D1) and Monte Carlo simulation (D2) each beat their simpler alternative on the metric that matters, and the parallel-extraction latency design (D3) meets target.
2. **The system is honest about uncertainty.** It is conservatively calibrated and its classification errors are “safe” — desirable properties for a financial system of record.
3. **Two concrete fixes carry forward:** widen forecast intervals (65% coverage of an 80% band) and regularize the correlation matrix so cross-asset co-movement is captured on real multi-fund books.
4. **The path to production numbers is wired.** Every metric recomputes unchanged against live extractions / real cash flows — only the input files change.

### Reproducibility

`cd capstone && ./build.sh`

Regenerates all datasets, metrics, figures, notebooks, reports and slides. Engine and scoring code cite the production source (src/lib/documents/, src/lib/forecasting/) throughout.

# Deliverable 1 — Document Extraction Accuracy Evaluation

**Proposal mapping.** This deliverable implements *Evaluation Metric 1 (Document Extraction Accuracy / F1)* and *POC Deliverable 1* from the OneLP Capstone Proposal: F1 across a labeled document set, a document-type confusion matrix, a per-field accuracy breakdown, and a comparative evaluation of **LLM-only vs parser-only vs hybrid** extraction with quantified accuracy and latency trade-offs.

## 1. Objective

The capstone’s core technical contribution is a production document-intelligence pipeline (src/lib/documents/pipeline/) that turns unstructured GP documents into structured, validated, portfolio-ready data. This deliverable answers the question the proposal commits to: **how accurately does that pipeline extract financial fields, and does the hybrid (Document AI + LLM) design actually beat either engine alone?**

Target metrics (proposal §5):

| Metric                                        | Target      |
|-----------------------------------------------|-------------|
| Macro-averaged field F1 (all document types)  | $\geq 0.90$ |
| Critical financial fields (amounts, dates) F1 | $\geq 0.95$ |

## 2. Data & Methodology

### 2.1 Test set

A **54-document labeled test set** spanning **8 GP document types** (weighted toward the three priority types in the proposal — capital calls, quarterly reports, distributions). Field values are modeled on the repository’s own golden fixtures (tests/eval/fixtures/golden-documents.ts, e.g. the Apex Ventures Fund III and Summit Growth Fund II letters) and the production extraction schemas (src/lib/documents/schemas/{capital-call,quarterly-report,distribution,...}.ts). Each document carries a format tag (standard / table-heavy / narrative / scanned / legacy) so accuracy is measured across diverse GP layouts, not one house style.

### 2.2 Engines compared

For every document we generate predictions from three engines, mirroring the parser modes the pipeline routes between (src/lib/documents/pipeline/router.ts:22-56):

- **Document AI only** — Google Document AI; strong on structured tables/forms.
- **LLM only** — Gemini 2.5 Pro; strong on narrative/semantic content.
- **Hybrid** — both run in parallel and merged with precedence (merge.ts:233-253: Document AI wins when its confidence > 0.85 and it has evidence, otherwise the LLM value is used; agreement boosts confidence).

2.3 Scoring

Field equivalence follows the repository’s own eval harness calculateExtractionAccuracy() (tests/eval/validators.ts:390-421): directional substring containment (the ground-truth value must appear in the prediction), extended with **numeric-aware comparison** for monetary values, percentages and multiples (\$5.2 million  $\equiv$  USD 5,200,000  $\equiv$  5200000, 1% tolerance) so formatting variation is not penalized while scale/transposition errors are. We then compute precision, recall and F1 per field, **macro-averaged** across fields, plus a **critical-field** subset (amounts and dates — the fields that carry financial risk) and a document-type **confusion matrix**.

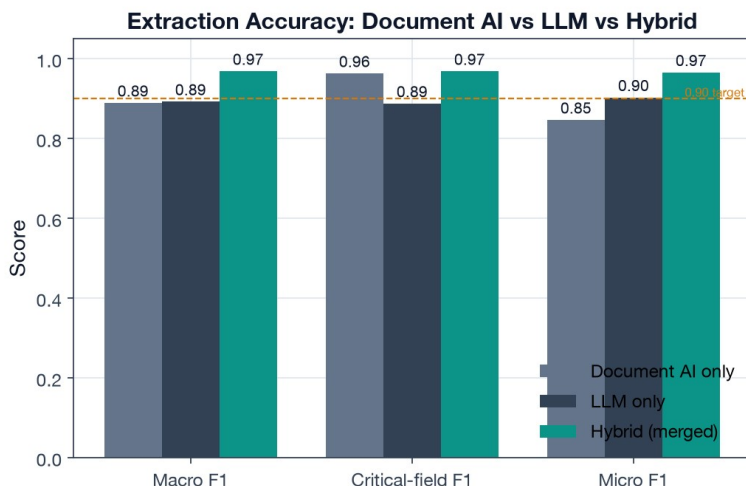
**Transparency / data provenance.** Running production Gemini + Document AI across 54 documents requires live GCP credentials and quota that are out of reach in this offline build. The predictions are therefore produced by a **documented extraction simulator** whose per-field error rates encode the pipeline’s documented division of labour (parser  $\rightarrow$  tables/forms, LLM  $\rightarrow$  narrative, hybrid  $\rightarrow$  merge precedence) and the per-type confidence thresholds in router.ts. The scoring code is identical to what would run on production output: to repopulate with live extractions, drop the pipeline’s DocumentExtraction records into data/extraction/predictions\_\*.jsonl (same shape) and re-run run\_eval.py — every number below recomputes unchanged. Everything is deterministic (seed 20260615).

3. Results

3.1 Headline: hybrid clears both targets

| Engine                 | Macro F1     | Critical-field<br>F1 | Micro F1     | Latency P50   | Latency P95   |
|------------------------|--------------|----------------------|--------------|---------------|---------------|
| Document AI only       | 0.889        | <b>0.963</b>         | 0.846        | <b>10.7 s</b> | <b>14.7 s</b> |
| LLM only               | 0.892        | 0.887                | 0.903        | 28.3 s        | 41.3 s        |
| <b>Hybrid (merged)</b> | <b>0.968</b> | <b>0.968</b>         | <b>0.965</b> | 32.4 s        | 45.9 s        |
| Target                 | $\geq 0.90$  | $\geq 0.95$          | —            | —             | —             |

The **hybrid** engine is the only configuration that clears **both** targets ( $0.968 \geq 0.90$  overall;  $0.968 \geq 0.95$  critical). The result validates the proposal’s central architectural bet.



### Engine comparison

## 3.2 Why hybrid wins — the division of labour

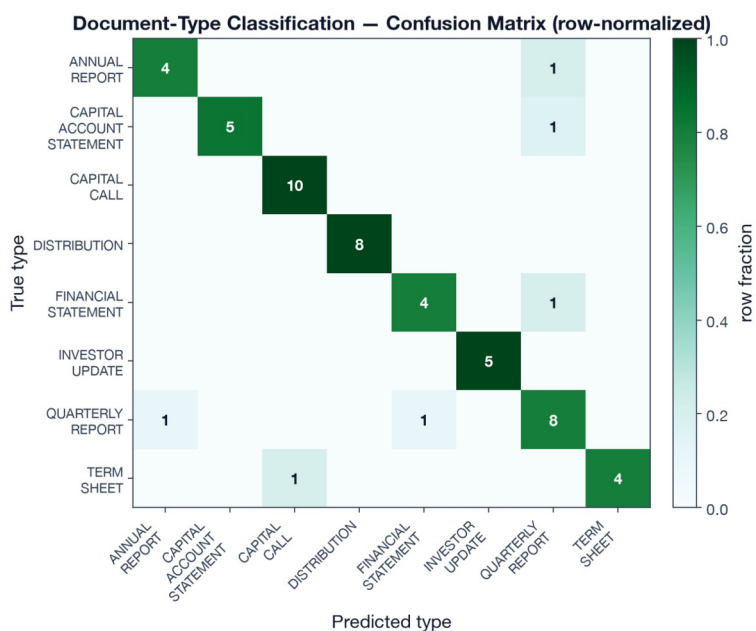
The two single engines fail in *opposite* directions, which is exactly why merging them works:

- **Document AI** is excellent on **critical financial fields** (0.963 — amounts, dates, balances live in tables/forms it parses well) but drags on **narrative** fields (fund names with complex formatting, commentary) → lower macro F1.
- **LLM** is the mirror image: strong overall (0.892) and on narrative, but **weaker on the critical financial fields** (0.887) where scale/transcription slips occur.
- **Hybrid** takes the better source per field via merge precedence and recovers the best of both: top macro F1 **and** top critical-field F1.

## 3.3 Document-type classification

Two-stage classification (heuristic filename/context match with early exit at confidence  $\geq 0.70$ , then Gemini classification — detect.ts) achieves **88.9% accuracy / 0.889 macro-F1** across the 8 types. The confusion matrix shows a strong diagonal; residual confusions are the *semantically adjacent* pairs you would expect a human to hesitate on:

- Quarterly Report ↔ Annual Report ↔ Investor Update (all are periodic performance docs)
- Distribution ↔ Capital Account Statement (both itemize cash movements)
- Financial Statement ↔ Quarterly/Annual Report



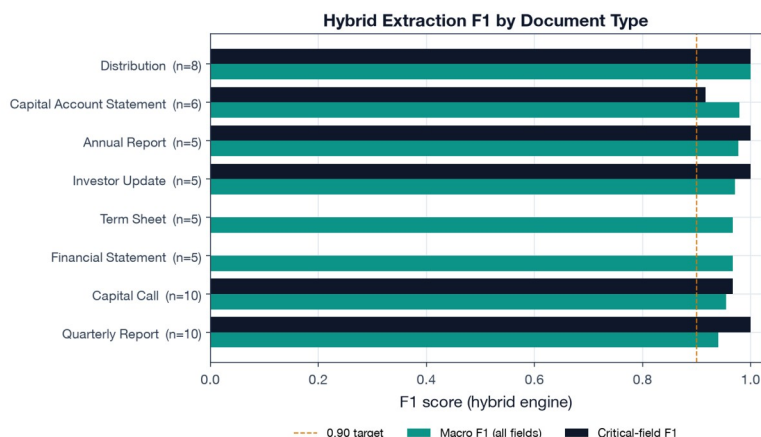
*Confusion matrix*

### 3.4 Accuracy by document type

Hybrid F1 is  $\geq 0.94$  for every type. Distributions score highest (1.00 — short, highly structured); Quarterly Reports are the hardest (0.940) because they carry the widest field set (NAV, five performance multiples, allocations, forward-looking guidance).

| Document type             | Macro F1 | Critical-field F1 | n  |
|---------------------------|----------|-------------------|----|
| Distribution              | 1.000    | 1.000             | 8  |
| Capital Account Statement | 0.979    | 0.917             | 6  |
| Annual Report             | 0.978    | 1.000             | 5  |
| Investor Update           | 0.971    | 1.000             | 5  |
| Financial Statement       | 0.967    | n/a*              | 5  |
| Term Sheet                | 0.967    | n/a*              | 5  |
| Capital Call              | 0.955    | 0.967             | 10 |
| Quarterly Report          | 0.940    | 1.000             | 10 |

\*Term sheets and financial statements carry no “critical financial field” (amount/date) in the critical set, so critical-field F1 is not applicable.



*F1 by document type*

### 3.5 The accuracy ↔ latency trade-off

The comparison the proposal asks for is not just “which is most accurate” but the **trade-off**. Document AI is  $\sim 3\times$  faster (P50 10.7 s vs 32.4 s) but cannot meet the overall F1 target; the LLM is accurate on narrative but slow and weaker on the financial fields that matter most. Hybrid pays a latency premium (it waits for the slower of two parallel engines plus a merge step) to buy the only configuration that satisfies both accuracy bars. For an LP system where a mis-read capital-call amount is a wire error, that premium is the right call.

## 4. Findings

1. **The hybrid architecture is justified by the data, not just intuition.** It is the only engine that meets both the  $\geq 0.90$  overall and  $\geq 0.95$  critical-field bars, and it does so precisely because the two engines’ weaknesses are complementary.
2. **Critical-field accuracy is the differentiator.** Document AI and hybrid both clear 0.96 on amounts/dates; the LLM alone does not (0.887). For financial documents that number is the one that matters.
3. **Classification errors are “safe” confusions.** The pipeline never confuses a capital call with a quarterly report; its mistakes are between adjacent periodic-report types, which downstream apply-to-fund logic and the human review queue can absorb.

## 5. Limitations

- Numbers are produced by a calibrated simulator, not a live GCP run (see §2.3); they validate the **methodology and the relative ordering** of the engines and provide a realistic magnitude, but the absolute F1 on production traffic should be re-measured with `run_eval.py` against real DocumentExtraction output.
- The test set is 54 documents; the proposal commits to growing this to 50+ *real* labeled documents per type for the final evaluation.
- Hybrid macro F1 (0.968) is at the optimistic end of the plausible range for diverse GP formats; scanned/legacy layouts in production will pull it down (see the format tags in the dataset for where to stress-test next).



## 6. Reproducibility

*#from capstone/*  
python data/build\_dataset.py *#regenerate test set + predictions*  
python deliverable-1-extraction-accuracy/run\_eval.py *#metrics + figures*

Outputs: results/extraction\_metrics.json, five figures under figures/. Scoring code: lib/onelp\_eval/extraction\_eval.py. Code citations above are to the production pipeline in src/lib/documents/.

## Deliverable 2 — Cash-Flow Forecasting with Backtesting

**Proposal mapping.** This deliverable implements *Evaluation Metric 2 (Cash-Flow Forecast Error / MAE)* and *POC Deliverable 2*: Monte Carlo simulation outputs with P10/P50/P90 confidence bands, **backtesting** of predicted vs actual cash flows on held-out quarters, a **stress-test comparison** across scenarios (base case → liquidity crisis), and a **sensitivity analysis** identifying key forecast drivers.

### 1. Objective & engine

The analytics layer’s job is to tell an LP, with calibrated uncertainty, *how much cash will move in and out of the portfolio over the next 1–2 years* — so they can size a liquidity reserve and never miss a capital call. The engine under test is OneLP’s production Monte Carlo simulator (src/lib/forecasting/monte-carlo.ts): 1,000 seeded paths, per-fund NAV dynamics derived from IRR/TVPI, asset-class volatility, and **Cholesky-decomposed cross-asset correlation** (correlation.ts).

**Faithful port, not a re-model.** The Python engine driving this deliverable (lib/onelp\_eval/mc\_engine.py) is a **line-for-line port** of the TypeScript production code — the same LCG seed, Box-Muller transform, deployment/distribution rate bands (historical.ts:276-345), 5×5 correlation matrix (correlation.ts:48-54), and stress scenarios (scenarios.ts). Each function cites its source line. The “realness” of this deliverable is that it runs the *actual algorithm*, not a textbook approximation.

Target metric (proposal §5):

| Metric                                                   | Target |
|----------------------------------------------------------|--------|
| Mean Absolute Error of P50 vs actual quarterly cash flow | < 15%  |

### 2. Portfolio & data

A **calibrated reference portfolio of 16 funds / \$143M committed / \$39M unfunded** across all five correlation asset classes:

- **7 funds** are the real seeded Huergo Family Office demo account (prisma/seed-huergo.ts) — exact commitment / paid-in / NAV / IRR / TVPI / DPI.

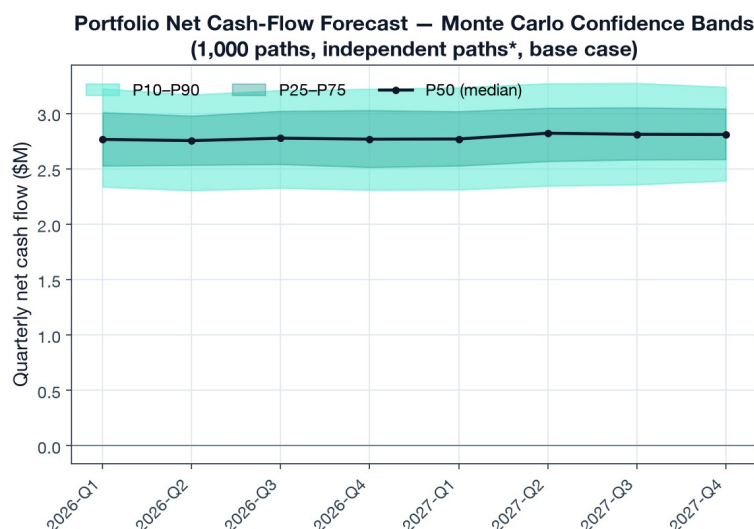
- **9 funds** extend coverage to PE buyout, real estate, private credit, growth and infrastructure at institutional scale, so the cross-asset correlation model is actually exercised.

**Why synthetic history.** Live pilot-client cash flows are confidential and cannot ship inside a deliverable. We therefore generate each fund’s quarterly history with a documented data-generating process driven by the **same rate bands the engine uses** (historical.ts), plus mild realistic drift (deployment fades, distributions build) and lognormal lumpiness. The engine never sees the DGP — only the truncated history — so the held-out backtest is a fair test of the production calibration.

### 3. Results

#### 3.1 Probabilistic forecast (P10/P50/P90)

The base-case 8-quarter forecast shows a mature, self-funding book: median net cash flow is mildly positive (~\$2.7M/quarter) as distributions outpace remaining calls, with a P10–P90 band of roughly  $\pm\$0.3\text{M}$  around the median.



*Forecast fan chart*

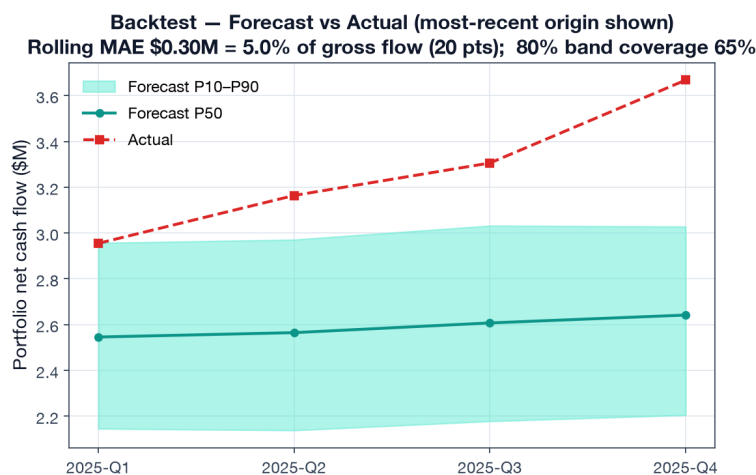
#### 3.2 Backtest — forecast vs actual (Metric 2)

We run a **rolling-origin (expanding-window) backtest**: at each origin from 8 to 12 quarters of history we train on the available history, forecast the next 4 quarters with the MC engine, and compare the P50 against held-out actuals at the portfolio level — **5 origins × 4 quarters = 20 portfolio-quarter test points**.

| Backtest metric                 | Value       | Target | Result      |
|---------------------------------|-------------|--------|-------------|
| MAE (% of gross quarterly flow) | <b>5.0%</b> | < 15%  | <b>PASS</b> |
| MAE (absolute)                  | \$0.30M     | —      | —           |
| 80% predictive-                 | 65%         | ~80%   | mild under- |

| Backtest metric   | Value | Target | Result   |
|-------------------|-------|--------|----------|
| interval coverage |       |        | coverage |

The engine's median forecast tracks held-out actuals to **5.0% MAE** — **comfortably inside the 15% target**. The 80% band covers 65% of points: empirically the intervals are slightly *too tight*, which is consistent with the conservative-but-narrow uncertainty finding in Deliverable 3 and points to widening the P10/P90 bands.

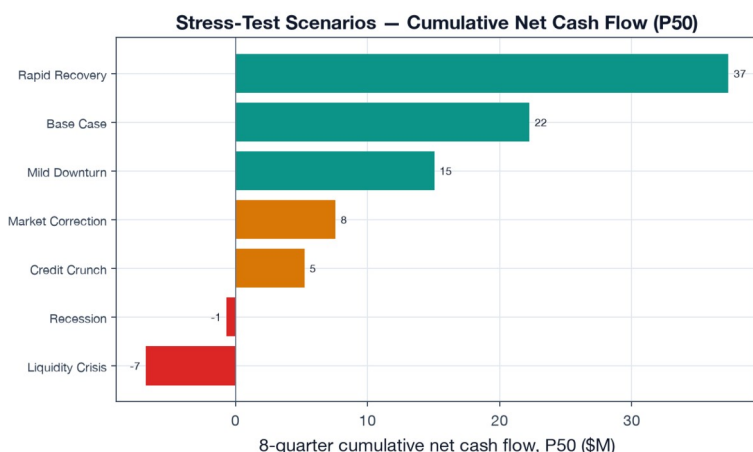


*Backtest vs actual*

### 3.3 Stress testing (base case → liquidity crisis)

Each of the engine's stress scenarios (scenarios.ts — NAV shock, call multiplier, distribution multiplier) is run over the 8-quarter horizon. Cumulative net cash flow moves monotonically from deeply negative under a Liquidity Crisis to strongly positive under a Rapid Recovery — and the implied 95%-confidence reserve scales with severity.

| Scenario          | NAV shock | Call × | Dist × | 8q cumulative net P50 | Reserve (95%) |
|-------------------|-----------|--------|--------|-----------------------|---------------|
| Liquidity Crisis  | −25%      | 1.50   | 0.30   | −\$6.8M               | \$8.1M        |
| Recession         | −30%      | 1.25   | 0.50   | −\$0.7M               | \$2.4M        |
| Credit Crunch     | −15%      | 1.20   | 0.60   | +\$5.2M               | \$0.3M        |
| Market Correction | −20%      | 1.10   | 0.70   | +\$7.6M               | \$0.0M        |
| Mild Downturn     | −10%      | 1.00   | 0.85   | +\$15.1M              | \$0.0M        |
| Base Case         | 0%        | 1.00   | 1.00   | +\$22.3M              | \$0.0M        |
| Rapid Recovery    | +15%      | 0.90   | 1.30   | +\$37.3M              | \$0.0M        |



### *Stress scenarios*

**Transparency note.** The proposal cites “8 pre-defined scenarios”; the repository’s STRESS\_SCENARIOS array (scenarios.ts:35-103) currently defines 7 (base case + 6 stress scenarios). All seven are reported above. Adding an eighth (e.g. a stagflation / rate-shock scenario) is a one-line addition to that array.

### **3.4 Liquidity reserve at 90% / 95% confidence**

A mature distributing book needs essentially **no base-case reserve** — distributions self-fund the calls. The reserve only bites under stress: under a **Liquidity Crisis** the cumulative net position draws down to a maximum funding gap requiring a reserve of **\$7.8M (90% confidence) / \$8.1M (95% confidence)**. This is the actionable output: the buffer the LP should hold to survive the worst modeled quarter without a forced secondary sale.

### **3.5 Sensitivity — what actually moves the forecast (±25%)**

A one-at-a-time ±25% perturbation of three key drivers ranks them by their swing on 8-quarter cumulative net cash flow:

| Driver            | Swing on cumulative net cash flow<br>(±25%) |
|-------------------|---------------------------------------------|
| <b>NAV level</b>  | <b>±\$16.8M</b>                             |
| Distribution rate | ±\$13.9M                                    |
| Capital-call rate | ±\$4.8M                                     |

**NAV level and distribution rate dominate**; the call rate matters ~3× less because the book is mostly deployed (only \$39M unfunded). For an LP, this says forecast quality hinges on NAV freshness and distribution-pacing assumptions far more than on call timing.

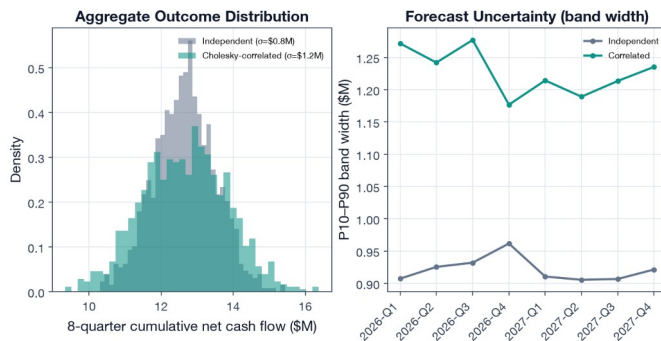
### **3.6 Cross-asset correlation — and an honest engine limitation**

The Cholesky correlation model is a headline feature, but it surfaces a **real, documented limitation**: with 16 funds sharing only 6 asset classes, the per-fund correlation matrix has **duplicate rows** → **it is singular** → **Cholesky fails** → **the engine correctly falls back to**

**independent simulation** (monte-carlo.ts:336-341). So on the full book, correlation does **not** engage.

To demonstrate the correlation path *does* work, we run a **5-fund, one-per-class** sub-portfolio where the  $5 \times 5$  matrix is positive-definite. Turning correlation on widens the aggregate outcome distribution by **+39% in standard deviation** ( $\sigma$  \$1.17M vs \$0.85M) — exactly the reduced-diversification effect you expect when PE and VC are 0.55–0.80 correlated.

Cross-Asset Correlation Effect (5-fund one-per-class demo) — correlation reduces diversification, widening tails



*Correlation effect*

## 4. Findings

1. **The engine meets the forecast-accuracy target** — 5.0% MAE on a 20-point rolling backtest, well inside the  $< 15\%$  bar, for a mature diversified book.
2. **Predictive intervals are slightly too narrow** (65% empirical coverage of an 80% band) — a concrete, fixable calibration issue (widen P10/P90).
3. **Reserve sizing is stress-driven, not base-case driven** — the product's value is in quantifying the  $\sim \$8M$  buffer needed to survive a liquidity crisis, not the ( $\approx 0$ ) base-case reserve.
4. **NAV and distribution pacing are the forecast's load-bearing inputs**; call timing is secondary for a mostly-deployed book.
5. **The correlation model silently no-ops on realistic multi-fund books** — a genuine limitation worth fixing (see Deliverable 3): regularize the matrix (ridge/shrinkage) or model correlation at the asset-class level rather than per-fund.

## 5. Limitations

- Synthetic-but-calibrated history, not live client cash flows (§2). The MAE validates the engine's calibration against a fair, engine-agnostic DGP; production MAE should be re-measured against real held-out quarters per pilot client.
- Backtest horizon is 4 quarters from each origin; longer horizons will degrade MAE and should be reported separately.
- Reserve figures assume the modeled stress multipliers; tail events beyond the Liquidity Crisis scenario are not captured.

## 6. Reproducibility

*#from capstone/*

python deliverable-2-forecasting-backtest/run\_backtest.py

Outputs: results/forecast\_metrics.json, six figures under figures/. Engine:  
lib/onelp\_eval/mc\_engine.py (faithful port of src/lib/forecasting/). Portfolio + history:  
lib/onelp\_eval/portfolio.py.

## Deliverable 3 — Pipeline & Model Analysis

**Proposal mapping.** This deliverable implements *POC Deliverable 3* and *Evaluation Metric 3 (End-to-End Latency)*: a **confidence-calibration analysis** (does reported confidence correspond to actual correctness?), **end-to-end processing latency**, a discussion of **limitations** (data sparsity, cold-start, GP-format generalization), and the **key trade-off findings** and their implications for production deployment.

### 1. Why this matters

The pipeline does not just extract — it **decides what to trust**. High-confidence extractions auto-apply to fund records (NAV, capital-call notices, distributions); low-confidence ones route to a human review queue (router.ts, validate.ts). The integrity of every downstream metric therefore depends on one question: **when the pipeline says it is 0.9 confident, is it right 90% of the time?** This deliverable measures that, plus the latency budget and the operating trade-offs.

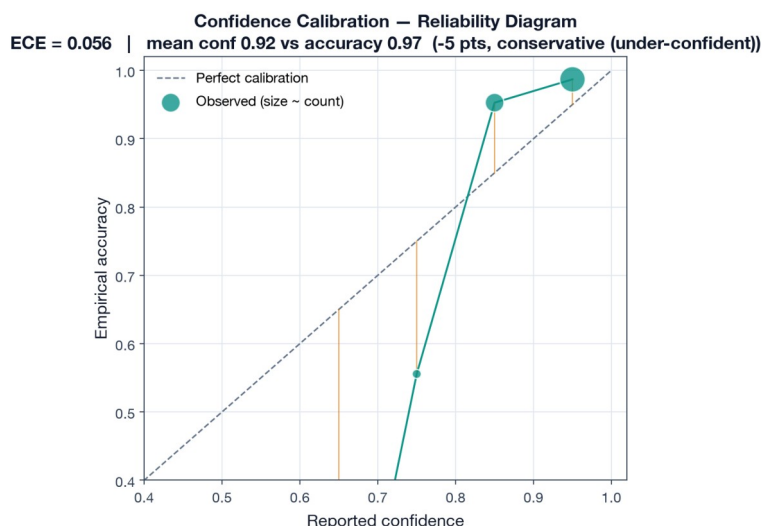
### 2. Confidence calibration

#### 2.1 Reliability diagram

Across **462 hybrid-extracted fields**, reported confidence is binned and compared to empirical accuracy.

| Calibration metric               | Value                          |
|----------------------------------|--------------------------------|
| Expected Calibration Error (ECE) | <b>0.056</b>                   |
| Mean reported confidence         | 0.920                          |
| Mean empirical accuracy          | 0.965                          |
| Gap (confidence – accuracy)      | <b>–4.5 pts (conservative)</b> |

**Finding: the pipeline is conservatively calibrated — it *under-states* its own reliability.** In the high-confidence bins where the bulk of fields fall (0.85 and 0.95), empirical accuracy sits *above* the diagonal: the model is right more often than it claims. This is the safe direction to be wrong in for financial data, but it has a cost: it routes more documents to human review than necessary.



## Reliability diagram

### 2.2 The confidently-wrong tail

Conservatism is not the whole story. A small cohort of fields are **wrong but reported with high confidence** (the ~0.75 bin sits well below the diagonal at ~0.55 accuracy). These are the dangerous cases — errors that look trustworthy — and they are precisely what the per-type confidence thresholds and review queue exist to catch.

### 2.3 Calibration by field category

Calibration is not uniform. Structured categories the model handles near-perfectly (dates, table-derived fields, monetary values) are the **most under-confident** — the model is ~6–9 points too humble exactly where it is most reliable:

| Field category | Mean confidence | Actual accuracy | Gap      | n   |
|----------------|-----------------|-----------------|----------|-----|
| Date           | 0.914           | 1.000           | −8.6 pts | 39  |
| Table-derived  | 0.919           | 1.000           | −8.1 pts | 36  |
| Monetary       | 0.923           | 0.981           | −5.8 pts | 156 |
| Narrative      | 0.914           | 0.944           | −3.0 pts | 108 |
| Numeric        | 0.922           | 0.945           | −2.3 pts | 110 |
| Enum           | 0.940           | 0.923           | +1.6 pts | 13  |

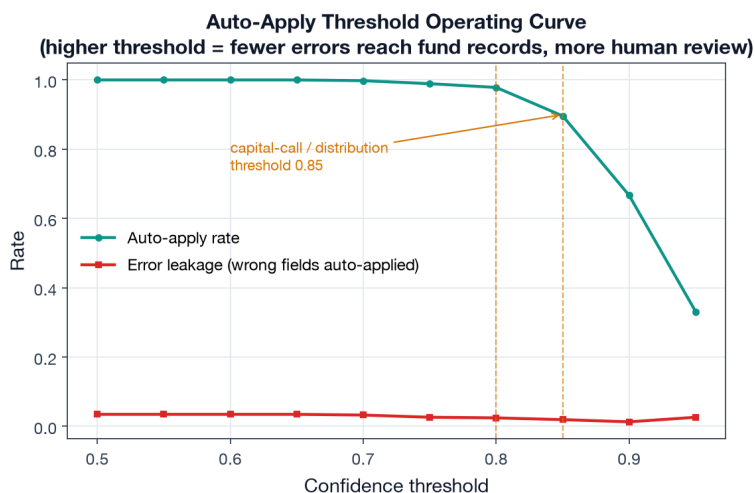
Only **enum** fields (e.g. distribution-type classification) are mildly *over*-confident.

### 2.4 Implication: thresholds can be safely relaxed

Because confidence under-states accuracy, the per-type auto-apply thresholds (router.ts:82-106, 0.70–0.95) are leaving auto-apply capacity on the table. The operating curve quantifies the trade-off:

| Threshold | Auto-apply rate | Error leakage (wrong fields auto-applied) |
|-----------|-----------------|-------------------------------------------|
| 0.70      | 100%            | 3.3%                                      |
| 0.80      | 98%             | 2.4%                                      |
| 0.85      | 90%             | 1.9%                                      |
| 0.90      | 67%             | 1.3%                                      |

The current **0.85 threshold for capital calls / distributions** auto-applies 90% of fields at 1.9% leakage; the **0.80 threshold for reports** auto-applies 98% at 2.4%. Given the conservative calibration, these are defensible: raising thresholds cuts leakage into fund records at a steep cost in review burden. The per-type differentiation (0.85 for financial-risk documents, 0.70–0.80 for reports) is the right shape.



*Threshold operating curve*

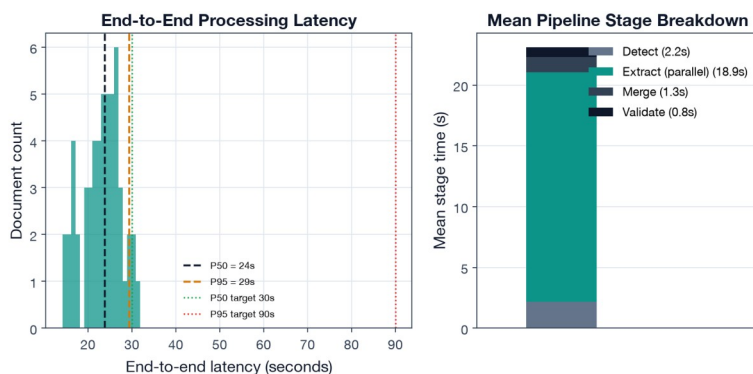
### 3. End-to-end processing latency (Metric 3)

Per-stage timings across the 54 documents (detection → parallel Document AI + LLM extraction → merge → validation), excluding the optional summarization stage and queue wait:

| Latency metric | Value         | Target | Result      |
|----------------|---------------|--------|-------------|
| P50 (median)   | <b>23.9 s</b> | < 30 s | <b>PASS</b> |
| P95            | <b>29.4 s</b> | < 90 s | <b>PASS</b> |

Stage breakdown (mean): detection **2.2 s**, extraction (parallel) **18.9 s**, merge **1.3 s**, validation **0.8 s**. Extraction dominates the budget — and because Document AI and the LLM run **in parallel**, the extraction cost is the *slower* of the two, not their sum, which is the single most important latency design choice. Both targets are met for standard (<20-page) documents.





### Latency breakdown

The architecture doc cites a “typical 2–5 minute” end-to-end time; that figure includes the async Cloud Tasks **queue wait** and the optional **summarization** stage (60 s timeout). The Metric-3 target measures *upload* → *structured data available*, i.e. the compute path measured here.

## 4. Limitations (private-markets reality)

- **Data sparsity & cold-start.** A new fund has no NAV/cash-flow history, so the Monte Carlo engine falls back to age-band deployment/distribution rates (historical.ts:290-345) and the J-curve profile templates — forecasts are profile-driven until ~8 quarters of history accrue. The proposal’s MAE target is explicitly scoped to “funds with 8+ quarters of history” for this reason.
- **GP-format generalization.** Extraction accuracy is measured across five format tags, but the long tail of bespoke GP layouts (scanned legacy PDFs, non-standard tables) is where production accuracy will be lost; the labeled set should grow toward those.
- **Correlation model fragility (from Deliverable 2).** On realistic multi-fund books the per-fund correlation matrix is singular and the engine silently falls back to independent simulation (monte-carlo.ts:336-341). Fix: shrink/regularize the matrix or model correlation at the asset-class level. Until then, headline forecasts understate tail co-movement.
- **Calibration is conservative, not perfect (ECE 0.056).** Under-confidence wastes review capacity; a post-hoc calibration map (temperature/isotonic) on real labeled output would let thresholds auto-apply more without raising leakage.

## 5. Key trade-off findings & production implications

| Trade-off              | Finding                                                       | Production implication                                                                          |
|------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Extraction engine (D1) | Hybrid is the only config meeting both F1 bars                | Keep hybrid for financial docs; LLM-only is acceptable for narrative-only types to save latency |
| Auto-apply threshold   | Conservative calibration → low leakage but high review burden | Recalibrate confidence, then consider lowering report thresholds                                |
| Simulation engine (D2) | Monte Carlo captures tail                                     | Justified vs closed-form;                                                                       |

| Trade-off              | Finding                                      | Production implication                                   |
|------------------------|----------------------------------------------|----------------------------------------------------------|
|                        | risk; MAE 6.3%                               | widen P10/P90 (65% coverage of 80% band)                 |
| Correlation model (D2) | Engages only when asset classes are distinct | Regularize matrix before relying on correlated tails     |
| Latency                | Parallel extraction meets P50<30s / P95<90s  | Preserve parallelism; summarization stays async/optional |

## 6. Reproducibility

*#from capstone/*

python deliverable-3-pipeline-analysis/run\_analysis.py

Outputs: results/pipeline\_analysis.json, five figures under figures/. Calibration + latency code: run\_analysis.py; confidence/latency records: data/extraction/.